

INTERNSHIP PROJECT REPORT

Project Title: Personal Expense Tracker

Domain: Web Development (Front-End)

ABSTRACT

The Personal Expense Tracker is a web-based application developed as part of my internship in the domain of Front-End Web Development. The main purpose of this project is to help users manage their daily expenses in a simple and organized way. In today's fast-moving world, people often spend money on various items but forget to keep track of their expenses. This project provides a convenient solution for recording, viewing, and managing expenses efficiently.

The application is built using basic front-end technologies such as HTML, CSS, JavaScript, jQuery, and Bootstrap 5. These technologies help in creating an interactive and responsive user interface that works smoothly on different devices like laptops, tablets, and mobile phones. The system allows users to add new expenses, view a list of expenses, and delete expenses when needed. The application also uses LocalStorage to save data in the browser so that the information remains available even after refreshing or reopening the page.

Through this project, I gained practical experience in designing web pages, handling user input, managing data storage, and improving user interface design. This project helped me understand the importance of front-end development in creating user-friendly applications. Overall, the Personal Expense Tracker is a simple yet useful tool that demonstrates the basic concepts of web development and data management.

OBJECTIVE

The main objective of developing the Personal Expense Tracker project is to create a simple and efficient system that helps users manage their daily expenses using web technologies. The project focuses on applying the knowledge gained during the internship in front-end web development.

The objectives of this project are as follows:

- To design and develop a user-friendly web application using HTML, CSS, JavaScript, jQuery, and Bootstrap.
 - To allow users to add, view, and delete expense records easily.
 - To store expense data using LocalStorage so that data is saved across browser sessions.
 - To create a responsive user interface that works properly on different screen sizes.
 - To provide a clear summary of expenses for better financial management.
 - To improve practical knowledge of front-end web development concepts.
 - To understand how to handle user input and dynamic data in web applications.
 - To develop problem-solving and logical thinking skills through project implementation.
-

INTRODUCTION

Web development is one of the most important fields in the technology industry today. Many businesses and individuals use web applications to perform daily tasks such as shopping, communication, banking, and financial management. Front-end web development focuses on designing and creating the visual part of websites that users interact with directly.

During my internship in the domain of Front-End Web Development, I learned the basics of HTML, CSS, JavaScript, jQuery, and Bootstrap. These technologies are widely used to build modern web applications. HTML is used to create the structure of a web page, CSS is used to design the layout and appearance, JavaScript is used to add functionality, jQuery simplifies JavaScript operations, and Bootstrap helps create responsive designs quickly.

One common problem faced by many people is managing their daily expenses. People often spend money on groceries, transportation, food, and other activities but do not maintain proper records. As a result, it becomes difficult to track spending and manage finances effectively. To solve this problem, the Personal Expense Tracker project was developed.

The Personal Expense Tracker is a simple web-based system that allows users to record their expenses and manage them easily. The application provides options to add new expenses, view the list of expenses, and delete unwanted entries. It also stores data using LocalStorage, which means the data remains available even after closing the browser.

This project demonstrates how front-end technologies can be used to build practical applications that solve real-life problems. It also helps students understand how different web technologies work together to create a complete system.

LITERATURE SURVEY

Before developing the Personal Expense Tracker system, I referred to various online learning platforms and documentation to understand the basic concepts of front-end web development. Websites such as W3Schools, MDN Web Docs, and Bootstrap documentation provided useful information about HTML structure, CSS styling, JavaScript functions, and responsive web design.

I also studied examples of simple expense tracking systems available on the internet. These systems helped me understand how users can add, store, and manage expense data effectively. By analyzing these examples, I was able to design a user-friendly interface and implement the required functionalities using front-end technologies.

The literature survey helped me gain knowledge about modern web development practices and improved my understanding of how real-world web applications are developed.

METHODOLOGY

The development of the Personal Expense Tracker project followed a step-by-step approach to ensure proper design, implementation, and testing of the system. The methodology used in this project is simple and suitable for beginner-level web development applications.

Step 1: Requirement Analysis

In this step, the requirements of the project were identified. The system needed to provide features such as adding expenses, viewing expenses, deleting expenses, and storing data using LocalStorage. The user interface also needed to be clean and responsive.

Step 2: System Design

After identifying the requirements, the structure of the web page was designed. The layout included input fields for entering expense details such as expense name, amount, and category. Buttons were added for adding and deleting expenses. A table or list was used to display the recorded expenses.

Step 3: Development

The system was developed using the following technologies:

HTML was used to create the basic structure of the web page.

CSS was used to design the layout, colors, fonts, and spacing.

JavaScript was used to implement the logic for adding and deleting expenses.

jQuery was used to simplify event handling and DOM manipulation.

Bootstrap was used to make the user interface responsive and visually attractive.

LocalStorage was used to store expense data in the browser.

Step 4: Testing

After development, the system was tested to ensure that all features were working correctly. Different test cases were performed, such as adding new expenses, deleting expenses, refreshing the page, and reopening the browser to check data persistence.

Step 5: Deployment

Finally, the application was executed in a web browser to verify its performance. The system successfully displayed expense records and stored data using LocalStorage.

SYSTEM ARCHITECTURE

The system architecture of the Personal Expense Tracker is based on a simple front-end design. The application runs completely in the web browser without requiring a server or database. All operations such as adding, deleting, and displaying expenses are handled using JavaScript.

The system consists of the following components:

- User Interface – Allows users to enter and view expense details
- JavaScript Logic – Processes user input and manages data
- LocalStorage – Stores expense data in the browser
- Web Browser – Displays the application and executes the code

This architecture makes the system simple, efficient, and easy to maintain.

TOOLS AND TECHNOLOGIES USED

HTML (HyperText Markup Language)

HTML is the standard language used to create web pages. It defines the structure and content of a webpage using elements such as headings, paragraphs, forms, and tables. In this project, HTML was used to create input fields, buttons, and the layout of the expense tracker.

CSS (Cascading Style Sheets)

CSS is used to design and style web pages. It controls the appearance of elements such as colors, fonts, spacing, and alignment. In this project, CSS was used to make the application visually attractive and easy to use.

JavaScript

JavaScript is a programming language used to add functionality to web pages. It allows developers to handle user input, perform calculations, and update the webpage dynamically. In this project, JavaScript was used to add, display, and delete expense records.

jQuery

jQuery is a JavaScript library that simplifies coding by providing easy methods for handling events and manipulating HTML elements. In this project, jQuery was used to manage user interactions efficiently.

Bootstrap 5

Bootstrap is a popular front-end framework used to create responsive and mobile-friendly web pages. It provides ready-made components such as buttons, forms, and navigation bars. In this project, Bootstrap was used to design the layout and improve responsiveness.

LocalStorage

LocalStorage is a web storage feature that allows data to be stored in the browser. The stored data remains available even after refreshing or reopening the webpage. In this project, LocalStorage was used to save expense records.

SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse
- Internet Connection (for development resources)

Software Requirements

- Web Browser (Google Chrome, Microsoft Edge, or Mozilla Firefox)
 - Code Editor (Visual Studio Code or Notepad++)
 - Operating System (Windows, Linux, or macOS)
-

SYSTEM IMPLEMENTATION

The implementation of the Personal Expense Tracker involves creating a web interface that allows users to manage expenses efficiently. The system consists of input fields for entering expense details and buttons for performing actions such as adding and deleting expenses.

When a user enters expense details and clicks the "Add Expense" button, the JavaScript function collects the data and stores it in LocalStorage. The stored data is then displayed in a table format on the webpage. If the user wants to remove an expense, they can click the "Delete" button, which removes the record from both the table and LocalStorage.

The application automatically loads saved expenses when the webpage is opened. This ensures that the user does not lose data after refreshing the page.

DATA FLOW DESCRIPTION

The data flow in the Personal Expense Tracker follows a simple process. When the user enters expense details, the system captures the input and processes it using JavaScript functions. The processed data is then stored in LocalStorage and displayed on the screen.

The main steps in data flow are:

1. User enters expense details

2. System validates the input
3. Data is stored in LocalStorage
4. Expense list is updated and displayed
5. User can delete records when required

This process ensures smooth functioning of the system.

FEATURES OF THE SYSTEM

The Personal Expense Tracker includes the following features:

- Add new expense records
 - View list of all expenses
 - Delete unwanted expenses
 - Store data using LocalStorage
 - Responsive user interface using Bootstrap
 - Simple and easy-to-use design
 - Fast performance and quick response
 - Data persistence across sessions
-

USER INTERFACE DESIGN

The user interface of the Personal Expense Tracker was designed to be simple and user-friendly. Bootstrap was used to create a responsive layout that works on different screen sizes such as mobile phones, tablets, and laptops.

The interface includes:

- Input fields for expense name and amount
- Dropdown menu for category selection
- Add Expense button
- Delete button
- Table to display expenses

The use of proper spacing, colors, and alignment makes the application visually attractive and easy to use.

WORKING OF THE SYSTEM

The working of the Personal Expense Tracker is simple and user-friendly. The user starts by opening the application in a web browser. The main page displays input fields where the user can enter expense details such as expense name, amount, and category.

After entering the details, the user clicks the "Add Expense" button. The system processes the input using JavaScript and stores the data in LocalStorage. The expense is then displayed in the list of expenses.

If the user wants to remove an expense, they can click the "Delete" button next to the record. The system immediately removes the record from the list and updates LocalStorage.

When the webpage is refreshed or reopened, the system automatically retrieves saved data from LocalStorage and displays it again. This ensures that the user's expense records are always available.

ADVANTAGES OF THE SYSTEM

The Personal Expense Tracker provides several advantages:

- Helps users manage daily expenses effectively
 - Saves time by organizing financial records
 - Easy to use and understand
 - No need for manual record keeping
 - Works on different devices and screen sizes
 - Stores data securely in the browser
 - Improves financial awareness and planning
-

LIMITATIONS OF THE SYSTEM

Although the system is useful, it has some limitations:

- Data is stored only in the browser
- Cannot be accessed from different devices
- No user authentication system
- Limited advanced features
- Requires manual data entry

These limitations can be addressed in future versions of the system.

FUTURE ENHANCEMENTS

The Personal Expense Tracker can be improved by adding more advanced features in the future. Some possible enhancements include:

- User login and authentication system
 - Cloud database integration
 - Expense category filtering
 - Monthly and yearly expense reports
 - Graphical representation of expenses
 - Export data to PDF or Excel
 - Mobile application version
-

TESTING AND VALIDATION

Testing is an important step in software development to ensure that the system works correctly. In this project, different test cases were performed to verify the functionality of the application.

The following tests were conducted:

- Checking whether expenses are added correctly
- Verifying that expenses are displayed properly

- Ensuring delete functionality works correctly
- Testing data persistence after refreshing the page
- Checking responsiveness on different devices

The system performed successfully during testing.

LEARNING OUTCOMES

Through the development of this project, I gained practical knowledge and hands-on experience in front-end web development. The project helped me understand how different technologies work together to create a functional web application.

The key learning outcomes include:

- Understanding HTML structure
 - Applying CSS styling
 - Implementing JavaScript logic
 - Using jQuery for event handling
 - Designing responsive layouts using Bootstrap
 - Storing data using LocalStorage
 - Improving problem-solving skills
-

INTERNSHIP EXPERIENCE SUMMARY

The internship in Front-End Web Development provided valuable learning opportunities and practical exposure to web technologies. During the internship, I learned the fundamentals of HTML, CSS, JavaScript, jQuery, and Bootstrap.

Working on the Personal Expense Tracker project helped me apply theoretical knowledge to real-world problems. The internship improved my technical skills, confidence in coding, and ability to develop web applications independently.

REFERENCES

- W3Schools – HTML, CSS, JavaScript Tutorials

- [Bootstrap Official Documentation](#)
 - [Mozilla Developer Network \(MDN\)](#)
 - [jQuery Documentation](#)
 - [Online Web Development Learning Resources](#)
-

CODE:-

```
<!DOCTYPE html>

<html>

  <head>

    <meta name="author" Content="Raksha">

    <meta name="description" contend="Expense Tracker Application">

    <title>Expense Tracker</title>

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">

    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.min.css">

  </head>

  <body>

    <nav class="navbar navbar-expand-lg navbar-light-bg bg-light">

      <div class="container-fluid">

        <a href="#" class="navbar-brand">
```

```

        <image src="image/logo.png" class="img-fluid"
alt="logo" width="35"></image>

        </a>

        <button type="button" class="navbar-toggler"
data-bs-toggler="collapse" data-bs-target="#navbar">

            <li class="bi bi-list"></li>

        </button>

        <div class="collapse navbar-collapse" id="navbar">

            <div class="navbar-nav ms ms-auto">Home</div>

        </div>

    </div>

</nav>

<div class="container mt-5">

    <div class="float-start">

        <label class="text-success"><b>Total Spent</b></label>

        <b><span id="totalSpent"></span></b>

    </div>

    <div class="float-end">

        <button type="button" class="btn btn-outline-primary"
onclick="showExpenseModal()">Add Expense</button>

    </div>

</div>

<div class="container py-5 px-5">

    <div class="d-flex justify-content-center">

```

```

<div class="col-sm-12 col-md-12 col-lg-12">

    <div class="card">

        <div class="card-body">

            <table class="table">

                <thead class="text-center">

                    <th>#</th>

                    <th>Description</th>

                    <th>Date</th>

                    <th>Spent Amount</th>

                </thead>

                <tbody class="text-center"
id="expenseTableBody">

                    </tbody>

                </table>

            </div>

        </div>

    </div>

</div>

<div class="modal fade" id="addExpenseModal"
data-bs-backdrop="static" data-bs-keyboard="false" tabindex="-1"
aria-labelledby="addExpenseModal" aria-hidden="true">

    <form id="addExpenseForm">

        <div class="modal-dialog">

```

```
<div class="modal-content">

    <div class="modal-header">

        <h5 class="modal-title"
id="addExpenseModalLabel">Add Expense</h5>

        <button type="button" class="btn-close"
data-bs-dismiss="modal" aria-label="close"></button>

    </div>

    <div class="modal-body">

        <div class="mb-1">

            <label for="addExpenseDescription"
class="form-label">Description</label>

            <textarea class="form-control"
id="addExpenseDescription" name="description" rows="3" placeholder="Add
Your Expense Description"></textarea>

        </div>

        <div class="mb-1">

            <label for="addExpenditureDate"
class="form-label">Expenditure Date</label>

            <input type="date" class="form-control"
id="addExpenditureDate" name="date">

        </div>

        <div class="mb-1">

            <label for="addExpenseAmount"
class="form-label">Spent Amount</label>

            <input type="number"
class="form-control" id="addExpenseAmount" name="spent"
placeholder="Add Your Spent Amount">

        </div>

    </div>

</div>
```

```

        <div class="modal-footer">

            <button type="button" class="btn
btn-secondary" data-bs-dismiss="modal">Cancel</button>

            <button type="button" class="btn
btn-primary" onclick="saveExpense()">Add Expense</button>

        </div>

    </div>

</div>

</form>

</div>

<script src="https://code.jquery.com/jquery-3.7.1.js"
integrity="sha256-eKhayi8LEQwp4NKxN+CfCh+3qOVUttJn3QNZ0TciWLP4="
crossorigin="anonymous"></script>

<script
src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.8/dist/umd/popper
.min.js"
integrity="sha384-I7E8VVD/ismYTF4hNIPjVp/Zjvgyol6VFvRkX/vR+Vc4jQkC+hVqc
2pM8ODewa9r" crossorigin="anonymous"></script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.min
.js"
integrity="sha384-G/EV+4j2dNv+tEPo3++6LCgdCROaejBqfUeNjuKAiuXbjrxilcCdd
z6ZAVfHWelY" crossorigin="anonymous"></script>

<script>

    getTotalExpenditure()

    createTableFromJSON()

    function showExpenseModal() {

        $("#addExpenseModal").modal('show');

    }

```

```
function saveExpense() {

    $("#addExpenseModal").modal('hide');

    var dataArr = $("#addExpenseForm").serializeArray();

    console.log(dataArr);

    var expenseObject = new Object();

    var storageObjectArr = [];

    var storageObject =
localStorage.getItem('expenseStorage');

    for(var i in dataArr){

        var name=dataArr[i]['name']

        var value=dataArr[i]['value']

        expenseObject[name]=value

    }

    console.log(expenseObject);

    if(storageObject !=null && storageObject !=undefined &&
storageObject !=''){

        storageObjectArr = JSON.parse(storageObject);

        storageObjectArr.push(expenseObject)

    }else{

        storageObjectArr.push(expenseObject)

    }

localStorage.setItem('expenseStorage',JSON.stringify(storageObjectArr))
;

    createTableFromJSON();

    $("#addExpenseModal").trigger('reset');
```



```

    }

    function createTableFromJSON() {

        var storageObjectArr = [];

        var storageObject =
localStorage.getItem('expenseStorage');

        var storageObjectArr = JSON.parse(storageObject);

        var html="";

        if(storageObject !=null && storageObject !=undefined &&
storageObject !=''){

            if(storageObjectArr && storageObjectArr.length >0){

                for(let i in storageObjectArr){

                    var date = new
Date(storageObjectArr[i]['date']);

                    date = date.toISOString();

                    date = date.split('T')[0]

                    html = html + '<tr>'

                                + '<td>'+(parseInt(i)+1) +

'</td>'

                                +

'<td>'+storageObjectArr[i]['description'] + '</td>'

                                + '<td>'+date+ '</td>'

                                +

'<td>'+storageObjectArr[i]['spent'] + '</td>'

                                + '<\\tr>'

                    }

                    console.log(html)

```

```

        }else{

            html='<tr><td colspan="4">No Expense Details
Added Yet</td></tr>'

        }

    }else{

        html='<tr><td colspan="4">No Expense Details
Added Yet</td></tr>'

    }

    $("#expenseTableBody").html(html)

}

function getTotalExpenditure() {

    var storageObjectArr = [];

    var storageObject =
localStorage.getItem('expenseStorage');

    var storageObjectArr = JSON.parse(storageObject);

    var spent=0;

    if(storageObject !=null && storageObject
!=undefined && storageObject !=''){

        if(storageObjectArr && storageObjectArr.length
>0) {

            for(let i in storageObjectArr){

                spent= parseInt(spent) +
parseInt(storageObjectArr[i]['spent'])

            }

        }else{

```

```
        spent=0;

    }

    }else{

        spent=0;

    }

    $("#totalSpent").html(spent)

}

</script>

</body>

</html>
```

OUTPUT:-

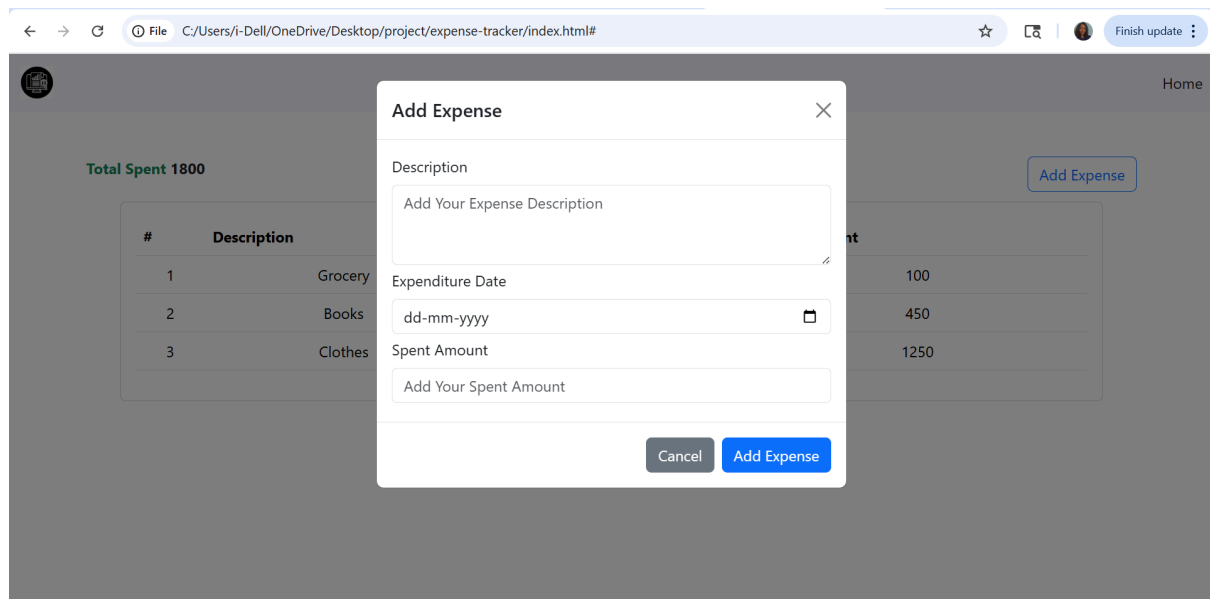
HTML ToDo List | Ma... x | HTML Calculator | Mi... x | Project Report Assis... x | Untitled document - x | Expense Tracker x | + | Ask Gemini | - | x

File C:/Users/i-Dell/OneDrive/Desktop/project/expense-tracker/index.html# ☆ | Finish update ⋮

Home

Total Spent 1800 [Add Expense](#)

#	Description	Date	Spent Amount
1	Grocery	2026-03-25	100
2	Books	2026-03-27	450
3	Clothes	2026-03-28	1250



CONCLUSION

The Personal Expense Tracker project successfully demonstrates the use of front-end web development technologies to build a practical application. The system allows users to record, view, and manage their expenses in a simple and efficient way. The use of HTML, CSS, JavaScript, jQuery, and Bootstrap helped in creating a responsive and user-friendly interface.

Through this project, I gained valuable hands-on experience in web development and improved my understanding of user interface design, data storage, and application functionality. The project also helped me develop problem-solving skills and confidence in building web applications.

Overall, the Personal Expense Tracker is a useful tool for managing daily expenses and learning the fundamentals of front-end development.
